

Naprawiamy grę SNAKE

Twoje pierwsze prawdziwe programowanie w Pythonie

Dostajesz kod gry Snake, który działa, ale jest mocno popsuty. Twoim zadaniem jest go naprawić — kawałek po kawałku. Każde zadanie ma swój numer, a w kodzie znajdziesz komentarz z tym samym numerem, np. `# ZADANIE 3`. Tam właśnie trzeba coś zmienić lub dopisać.

Złota zasada: po każdym zadaniu zapisz plik (Ctrl+S) i uruchom grę przyciskiem ► Run. Od razu zobaczysz, co się zmieniło!

0 Zanim zaczniesz — uruchom popsutą grę *5 minut*

Jeśli komputer jest już przygotowany, wykonaj tylko punkty 3–5. Pełne instrukcje są też w pliku README.

1. Zainstaluj Pythona (python.org → Download; zaznacz „Add python.exe to PATH”).
2. Zainstaluj Visual Studio Code (code.visualstudio.com → Download).
3. Otwórz folder gry: w VS Code File → Open Folder... → wybierz folder snake-warsztaty. Zgódź się na „trust” i zainstaluj proponowane rozszerzenie Python.
4. Zainstaluj silnik gry: menu Terminal → Run Task... → Zainstaluj pygame.
5. Kliknij plik snake.py po lewej, a potem ► Run w prawym górnym rogu.

Sprawdź efekt: otwiera się różowe okno z czerwoną kropką. Brzydkie? Bardzo. Zaraz to naprawimy.

1 Twoja gra — Twój tytuł

Znajdź w kodzie komentarz `# ZADANIE 1`. Linijka pod nim ustawia napis na belce okna. Napis (programiści mówią: string) zawsze stoi w cudzysłowach `"..."`. Zamień `???` na własny tytuł gry — jaki tylko chcesz!

```
pygame.display.set_caption("Super Snake 3000")
```

Sprawdź efekt: na górnej belce okna gry pojawia się Twój tytuł.

2 Ratuj oczy! Kolor tła

Kolory w grach to trzy liczby od 0 do 255: (czerwony, zielony, niebieski) — po angielsku RGB. Np. `(255, 0, 255)` to maksimum czerwonego + maksimum niebieskiego = ten okropny róż. Znajdź `# ZADANIE 2` i wpisz inne liczby. Wypróbuj kilka pomysłów:

<code>(25, 40, 25)</code>	ciemna zieleń trawy
<code>(10, 15, 45)</code>	granat nocnego nieba
<code>(0, 0, 0)</code>	czerń
<code>(40, 25, 10)</code>	ciemny brąz ziemi

Albo wymyśl własny kolor! Eksperymentuj do woli — zepsuć się nie da.

Sprawdź efekt: tło gry ma kolor, który sam wybrałeś/wybrałaś.

3 Gdzie jest wąż?! *najważniejsza naprawa*

Wąż w tej grze to lista krutek o nazwie snake. Żeby go narysować, pętla for ma przejść po kolei przez wszystkie kratki z listy. Ale ktoś napisał `range(0, 0)` — czyli „licz od zera do zera”. Taka pętla wykona się zero razy i wąż w ogóle się nie rysuje!

Napraw pętlę przy # ZADANIE 3 tak, żeby liczyła od zera do długości węża. Długość listy podaje `len(snake)`:

```
for i in range(len(snake)):
```

Sprawdź efekt: wąż się pojawia! No dobrze... pojawiają się trzy malutkie okruszki. Tym zajmujemy się za chwilę.

4 Wąż na siłowni

Każdy kawałek węża to prostokąt rysowany funkcją `pygame.draw.rect`. Dwie ostatnie liczby w nawiasie to jego szerokość i wysokość w pikselach. Teraz jest tam 4, 4 — stąd okruszki. Jedna kratka planszy ma rozmiar CELL (czyli 25 pikseli).

Przy # ZADANIE 4 zamień oba 4 na `CELL - 1`. (To „minus jeden” zostawia cieniutką przerwę między kratkami — dzięki temu widać kawałki węża. Sprawdź, co się stanie z samym CELL!)

```
pygame.draw.rect(screen, (90, 200, 70),  
                  (x * CELL, y * CELL, CELL - 1, CELL - 1))
```

Sprawdź efekt: wąż jest wreszcie gruby i porządnym.

5 To wąż czy ślimak?

Na samym dole pętli gry jest linijka `clock.tick(2)` przy # ZADANIE 5. Liczba w nawiasie mówi, ile kroków na sekundę robi wąż. Dwa kroki na sekundę to żenada.

Wpisz 9. A potem dla zabawy sprawdź 60 — da się w ogóle grać? Na koniec ustaw taką prędkość, przy której gra jest fajna.

Sprawdź efekt: wąż śmiga w normalnym tempie.

6 Pełne sterowanie *tu piszesz najwięcej kodu*

Spróbuj posterować strzałkami. Działa tylko strzałka w górę! Kierunek węża to dwie zmienne: `dx` (o ile w prawo) i `dy` (o ile w dół). Uwaga, pułapka: na ekranie oś y rośnie W DÓŁ, więc ruch w górę to `dy = -1`.

Znajdź # ZADANIE 6. Pod gotowym `if` od strzałki w górę dopisz trzy kolejne warunki — przepisz ten szkielecik i sam(a) uzupełnij znaki zapytania. Pilnuj wcięć: `elif` zaczyna się dokładnie pod `if`!

```
elif event.key == pygame.K_DOWN and dy != -1:  
    dx, dy = ?, ?  
elif event.key == pygame.K_LEFT and dx != 1:  
    dx, dy = ?, ?  
elif event.key == pygame.K_RIGHT and dx != -1:  
    dx, dy = ?, ?
```

Podpowiedź: w prawo = 1, 0, w górę = 0, -1. A ta druga część warunku (np. `dy != -1`) pilnuje, żeby wąż nie mógł zawrócić o 180 stopni i wjechać sam w siebie.

Sprawdź efekt: sterujesz wężem we wszystkie cztery strony.

7 Obiad! Zjadanie jabłka

Wąż przechodzi przez jabłko, jakby go nie było. Znajdź # ZADANIE 7. Jest tam warunek `if False`: — czyli „nigdy”. Zamień `False` na pytanie: czy głowa węża jest w tym samym miejscu co jabłko? W Pythonie „czy równe?” zapisujemy podwójnym znakiem `==`:

```
if head == apple:
```

Potem skasuj słowo `pass` i w jego miejscu napisz, co ma się stać po zjedzeniu: punkty rosną o 1, a jabłko przeskakuje w nowe, losowe miejsce. Obie linijki muszą mieć wcięcie 4 spacji (tak jak miało `pass`):

```
score = score + 1
apple = (random.randint(0, COLS - 1), random.randint(0, ROWS - 1))
```

`random.randint(0, COLS - 1)` losuje liczbę — jak rzut kostką, tylko od 0 do numeru ostatniej kolumny.

Sprawdź efekt: wąż zjada jabłko, ROBI SIĘ DŁUŻSZY, a nowe jabłko pojawia się gdzie indziej.

8 Ściany! Koniec gry

Teraz wąż spokojnie wyjeżdża poza ekran i znika na zawsze. Gra powinna się kończyć, gdy głowa wyjdzie poza planszę. Plansza ma `COLS` kolumn (numery 0–23) i `ROWS` wierszy (numery 0–17).

Znajdź # ZADANIE 8. Pod linijką `x, y = head` dopisz warunek — słowo `or` znaczy „lub”, a puste miejsca uzupełnij `sam(a)`:

```
if x < 0 or x >= ____ or y < 0 or y >= ____:
    running = False
```

Podpowiedź: co pasuje w luki — ile kolumn i wierszy ma plansza? Nazwy tych „pudełek z liczbami” są na samej górze kodu.

Sprawdź efekt: uderzenie w ścianę = ekran „KONIEC GRY” z Twoim wynikiem.

9 Uczciwy licznik punktów

Licznik w rogu zawsze pokazuje 0, bo ktoś wpisał zero na sztywno w napis. Znajdź # ZADANIE 9. Prawdziwe punkty siedzą w zmiennej `score`, ale to liczba, a wyświetlić trzeba napis. Funkcja `str(...)` zamienia liczbę na napis, a plus skleja dwa napisy:

```
text = font.render("Punkty: " + str(score), True, (255, 255, 255))
```

Sprawdź efekt: licznik pokazuje prawdziwy wynik. Gra jest GOTOWA — zagraj i pobij rekord kolegi lub koleżanki!



Zadania bonusowe — dla mistrzów

A. Wąż gryzie własny ogon. Prawdziwy Snake kończy się też wtedy, gdy wąż wjedzie w siebie. Dopisz pod warunkiem ze ścianami (zadanie 8):

```
if head in snake[1:]:  
    running = False
```

(in pyta: „czy głowa jest gdzieś na liście reszty ciała?”)

B. Coraz szybciej! Niech gra przyspiesza z każdym zjedzonym jabłkiem. Zamień linijkę z zadania 5 na:

```
clock.tick(9 + score // 3)
```

C. Twój styl. Zmień kolor węża i jabłka na własne (wiesz już, gdzie szukać trójek liczb!).

D. Gigantyczna plansza. Na górze kodu zmień COLS i ROWS. Czy warunek ze ścianami dalej działa? Dlaczego?

Gratulacje! Naprawiłeś/naprawiłaś prawdziwą grę prawdziwym kodem w Pythonie. Ten sam język napędza YouTube, Instagram i Spotify.